

SQL - Structured Query Language & Database

Book 4

MODCOM Institute of Technology

Table of Contents

What Is a Database?....4
Database defined....4
What's the difference between a database and a spreadsheet?....4
What are databases used for?....5
Example of How Database Works with Python....6
SQL - Structured Query Language....7
What You Can Do with SQL....7
What's Covered....8
SQL Getting Started....8
Getting Started with SQL....9
Linux Installation....9
Windows Installation....9
Open Xampp in Linux....9
Open Xampp On Windows....10
Creating a Database....11
Creating a Table....11
Data Types....12
Adding Primary Key....15
SQL QUERY....16
Inserting Data in Table....17
SQL INSERT Statement....17
Syntax....17
SQL SELECT Statement....20
Selecting Data from Table....20
Syntax....20
Select All from Table....22
Example....22
Select Columns from Table....23
Example....23
SQL WHERE Clause....24
Selecting Record Based on Condition....24
Syntax....24
Filter Records with WHERE Clause....25
Example....25
Example....26
Example....27
Operators Allowed in WHERE Clause....27
SQL AND & OR Operators....28
Selecting Record Based on Condition....28
The AND Operator....28
Using WHERE Clause with AND Operator....29
Example....30

MODCOM Institute of Technology

The OR Operator....30
Example....30
Combining AND & OR Operator....31
Example....31
SQL IN & BETWEEN Operators....32
Working with Range and Membership Conditions....32
The IN Operator....32
Example....33
Example....34
The BETWEEN Operator....34
Define Numeric Ranges....35
Example....35
SQL ORDER BY Clause....36
Ordering the Result Set....36
Syntax....36
Sorting Single Column....37
Example....37
Example....37
Example....38
SQL UPDATE Statement....39
Updating Table Data....39
Syntax....39
Updating a Single Column....41
Example....41
Updating Multiple Columns....42
Example....42
SQL DELETE Statement....43
Deleting Data from Tables....43
Syntax....43
Delete Records Based on Conditions....44
Example....44
Delete All Data....45
Example....45
References....46

MODCOM Institute of Technology

What Is a Database?

Database defined

A database is an organized collection of structured information, or data, typically stored electronically in a computer system. A database is usually controlled by a database management system (DBMS). Together, the data and the DBMS, along with the applications that are associated with them, are referred to as a database system, often shortened to just database.

Data within the most common types of databases in operation today is typically modeled in rows and columns in a series of tables to make processing and data querying efficient. The data can then be easily accessed, managed, modified, updated, controlled, and organized. Most databases use structured query language (SQL) for writing and querying data.

What's the difference between a database and a spreadsheet?

Databases and spreadsheets (such as Microsoft Excel) are both convenient ways to store information. The primary differences between the two are:

1. How the data is stored and manipulated
2. Who can access the data
3. How much data can be stored

Spreadsheets such as Excel were originally designed for one user, and their characteristics reflect that. They're great for a single user or small number of users who don't need to do a lot of incredibly complicated data manipulation. Databases, on the other hand, are designed to hold much larger collections of organized information—massive amounts, sometimes. Databases allow multiple users at the same time to quickly and securely access and query the data using highly complex logic and language.

MODCOM Institute of Technology

What are databases used for?

Businesses use data stored in databases to make informed business decisions. Some of the ways organizations use databases include the following:

- **Improve business processes.** Companies collect data about business processes, such sales, order processing and customer service. They analyze that data to improve these processes, expand their business and grow revenue.

- **Keep track of customers.** Databases often store information about people, such as customers or users. For example, social media platforms use databases to store user information, such as names, email addresses and user behavior. The data is used to recommend content to users and improve the user experience.

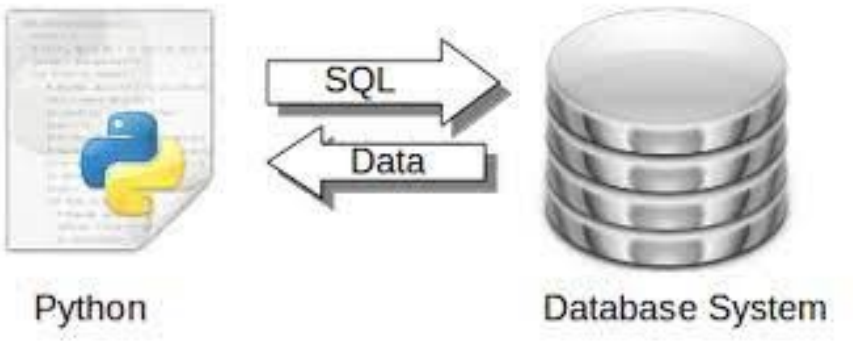
- **Secure personal health information.** Healthcare providers use databases to securely store personal health data to inform and improve patient care.

- **Store personal data.** Databases can also be used to store personal information. For example, personal cloud storage is available for individual users to store media, such as photos, in a managed cloud.

Check PDF - 10 Uses of a Database From https://coding.co.ke/content/SQL/

MODCOM Institute of Technology

Example of How Database Works with Python



MODCOM Institute of Technology

SQL - Structured Query Language

SQL is a standard language designed for managing data in relational database management system.

SQL stands for Structured Query Language. SQL is a standard programming language specifically designed for storing, retrieving, managing or manipulating the data inside a relational database management system (RDBMS).

SQL became an ISO standard in 1987.

SQL is the most widely-implemented database language and supported by the popular relational database systems, like MySQL, SQL Server, and Oracle. However, some features of the SQL standard are implemented differently in different database systems.

SQL was originally developed at IBM in the early 1970s. Initially it was called SEQUEL (Structured English Query Language) which was later changed to SQL (pronounced as S-Q-L).

What You Can Do with SQL

There are lot more things you can do with SQL:

•You can create a database.

•You can create tables in a database.

•You can query or request information from a database.

•You can insert records in a database.

•You can update or modify records in a database.

MODCOM Institute of Technology

•You can delete records from the database.

•You can set permissions or access control within the database for data security.

•You can create views to avoid typing frequently used complex queries.

The list does not end here, you can perform many other database-related tasks with SQL. You will learn about all of them in detail in upcoming chapters.

What’s Covered.

This SQL series covers all the fundamental concepts of SQL language, such as creating database and tables, using constraints, adding records to a table, selecting records from a table based on different conditions, updating and deleting records in a table, and so on.

Once you're familiar with the basics, you'll move on to next level that explains the methods of retrieving records through joining multiple tables, searching records in the table based on pattern, etc.

Finally, you'll explore some advanced concepts, such as modifying the existing table structure, performing aggregations and grouping data together, creating views to simplify and secure table access, creating indexes for tables, handling dates and times, cloning of an existing table, creating temporary tables, as well as, concept of SQL subqueries and SQL injection.

SQL Getting Started

In this Lesson you will learn how to set up a SQL practice environment.

MODCOM Institute of Technology

Getting Started with SQL

As you already know SQL is used to communicate with the database, so before you start experimenting with SQL, you need access to a database system first.

To execute some SQL statement you'll need access to a full-fledged database management system like MySQL, SQL Server etc.
In this class we will use MySQL.

To use Xampp, it must be installed from
https://www.apachefriends.org/download.html

Linux Installation.

https://www.wikihow.com/Install-XAMPP-on-Linux

Windows Installation

https://www.wikihow.com/Install-XAMPP-for-Windows

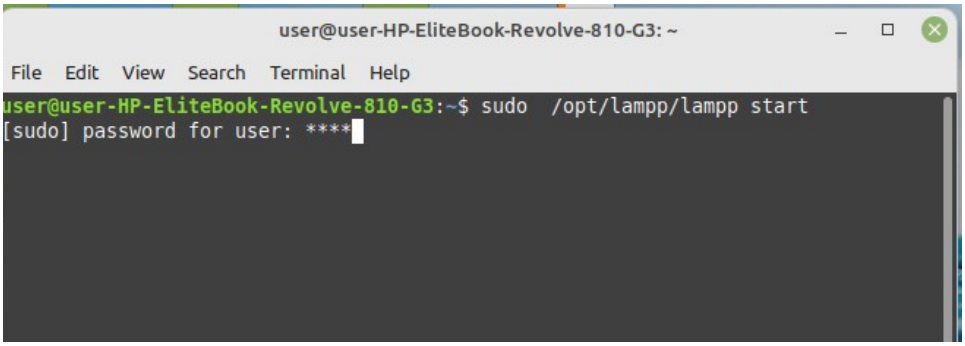
Once xampp is installed. Follow below step to start it.

Open Xampp in Linux

Open Terminal and type Below Command.

sudo /opt/lampp/lampp start

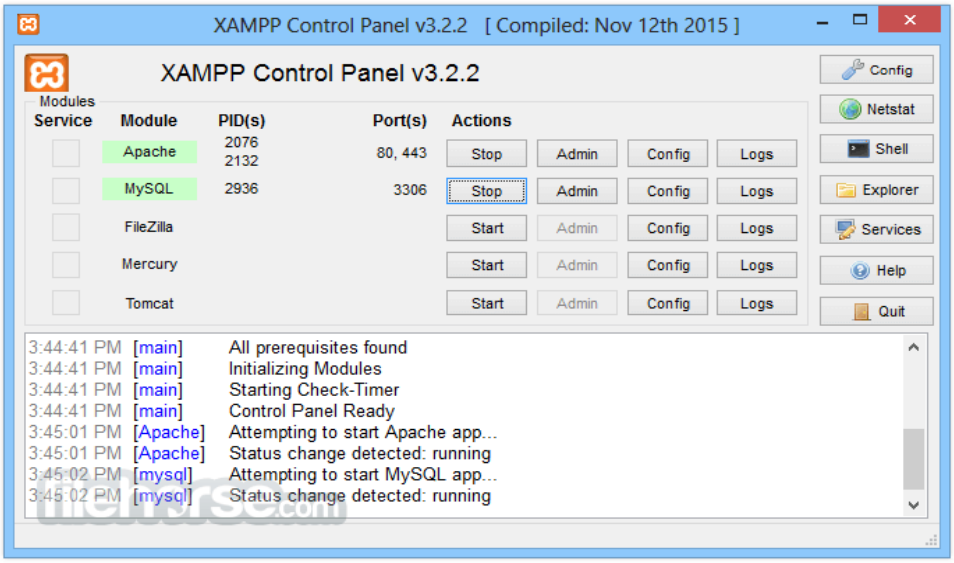
Press Enter and Type Your PC Password.



MODCOM Institute of Technology

Open Xampp On Windows

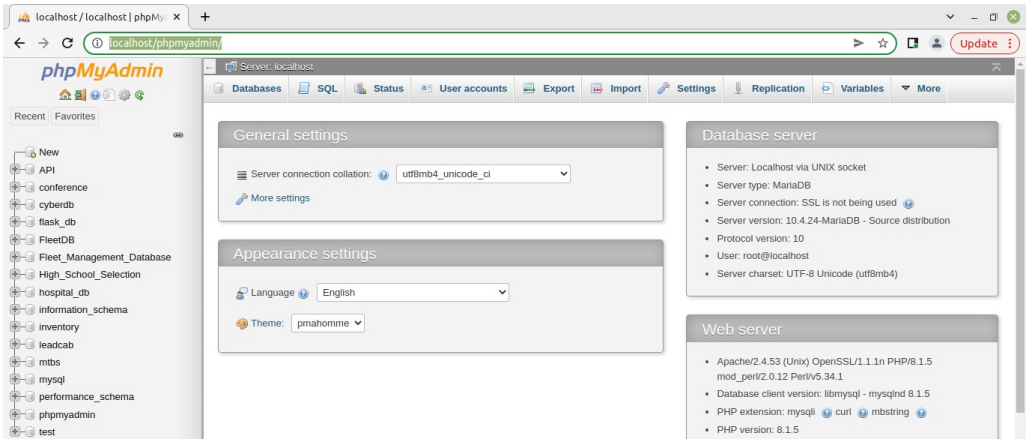
On Xampp Control Panel, Start both Apache and MySQL.



Open a browser using either Chrome or Firefox or any other browser.

http://localhost/phpmyadmin/

The next screen that appears looks like below.



Done. You are now ready to use MySQL.

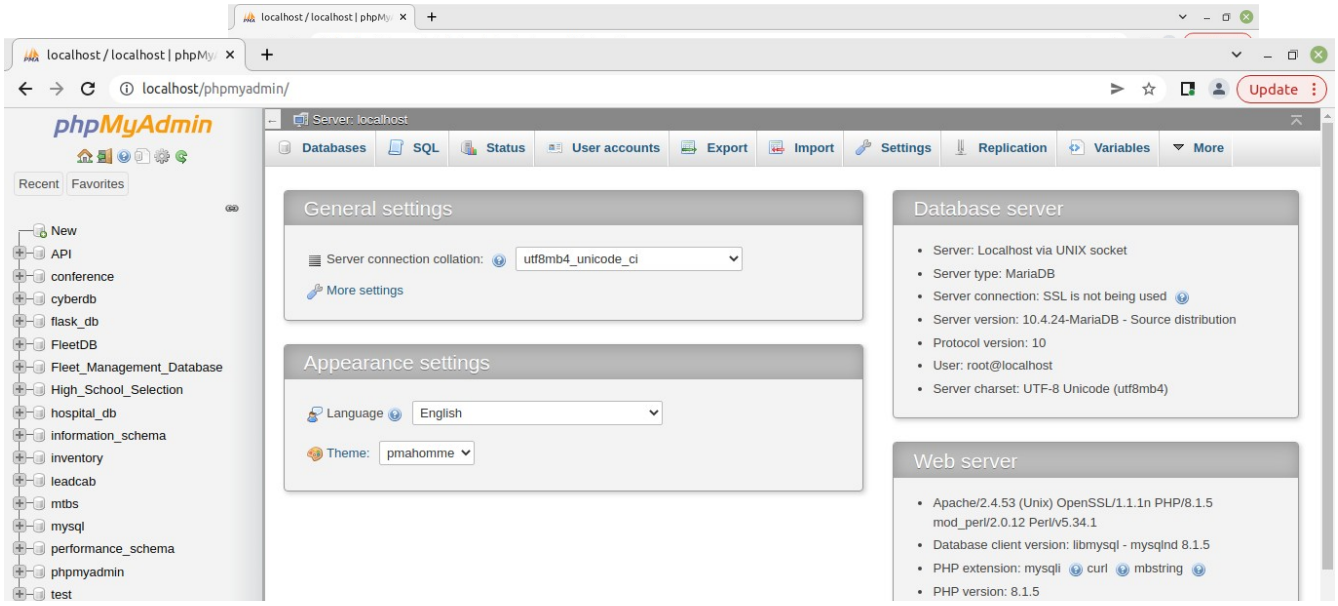
MODCOM Institute of Technology

Creating a Database.

Before doing anything with the data we must need to create a database first. We're assuming that you already have a MySQL, server (Xampp) available for your use, as well as you've all the necessary privileges.

Under this Page Click on **Databases**.

You'll get to below screen. Type database name and **Create**



Your database is now Created.

Creating a Table

In the previous chapter we have learned how to create a database on the database server. Now it's time to create some tables inside our database that will actually hold the data. A database table simply organizes the information into rows and columns.

MODCOM Institute of Technology

There are a few additional constraints (also called modifiers) that are set for the table columns in the preceding statement. Constraints define rules regarding the values allowed in columns.

•The `NOT NULL` constraint ensures that the field cannot accept a `NULL` value.

•The `PRIMARY KEY` constraint marks the corresponding field as the table's primary key.

•The `AUTO_INCREMENT` attribute is a MySQL extension to standard SQL, which tells MySQL to automatically assign a value to this field if it is left unspecified, by increment the previous value by 1. Only available for numeric fields.

•The `UNIQUE` constraint ensures that each row for a column must have a unique value.

Data Types

The following table summarizes the most commonly used data types supported by MySQL.

Data Type	**Description**
INT	Stores numeric values in the range of -2147483648 to 2147483647
DECIMAL	Stores decimal values with exact precision.
CHAR	Stores fixed-length strings with a maximum size of 255 characters.
VARCHAR	Stores variable-length strings with a maximum size of 65,535 characters.
TEXT	Stores strings with a maximum size of 65,535 characters.

MODCOM Institute of Technology

Data Type	**Description**
DATE	Stores date values in the YYYY-MM-DD format.
DATETIME	Stores combined date/time values in the YYYY-MM-DD HH:MM:SS format.
TIMESTAMP	Stores timestamp values. `TIMESTAMP` values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:01' UTC).

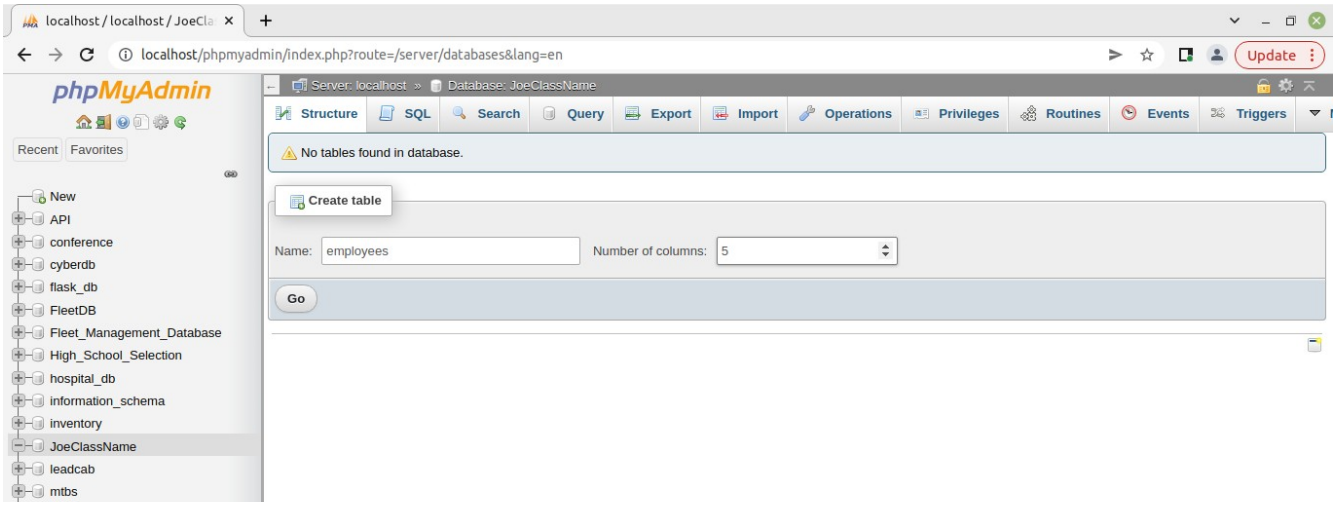
We now need to create a table, a table has rows and columns. Each columns must have a data type to specify the the data to be entered.

Example table Employees Data table

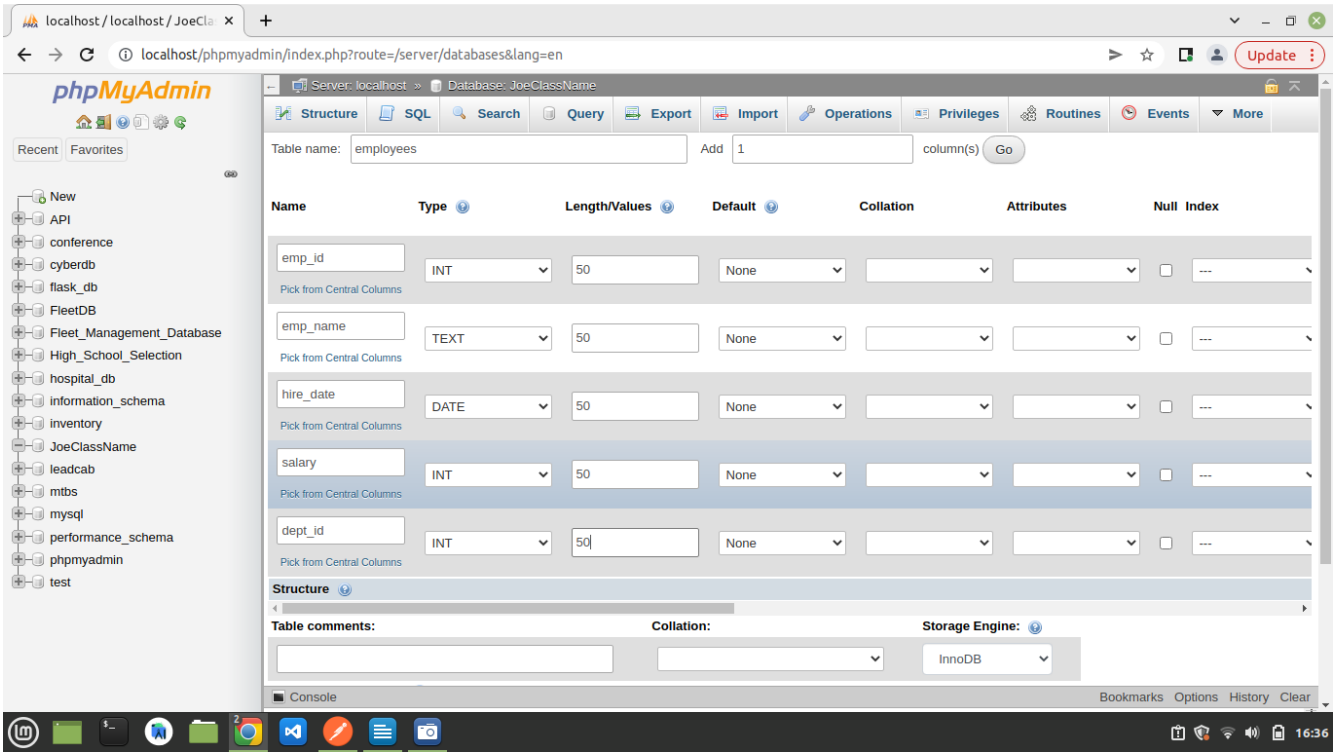
emp_id	INT(50) (Primary Key, Auto Increment)
emp_name	TEXT
hire_date	DATE
salary	INT(50)
dept_id	INT(50)

To Create above table in Xampp Click on your database.

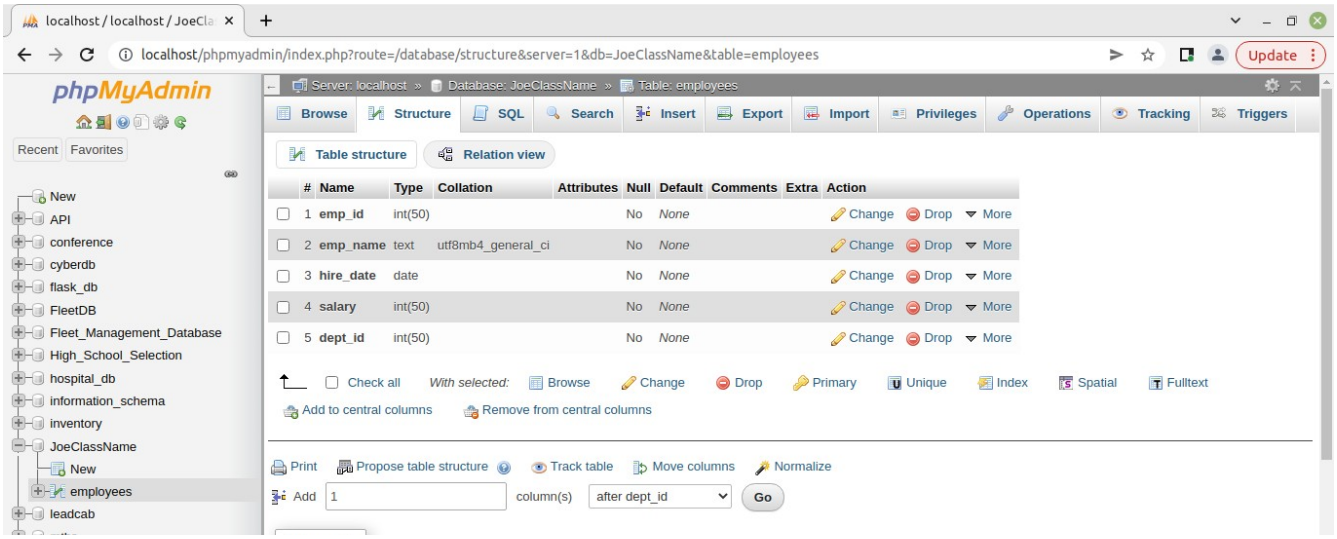
Enter the table name '**employees**', and Number of columns enter 5, then **Go**



The next screen shows where you enter the 5 columns names, data type and length.



Once done click save at the Bottom. Then below screen shows the table created and its structure.

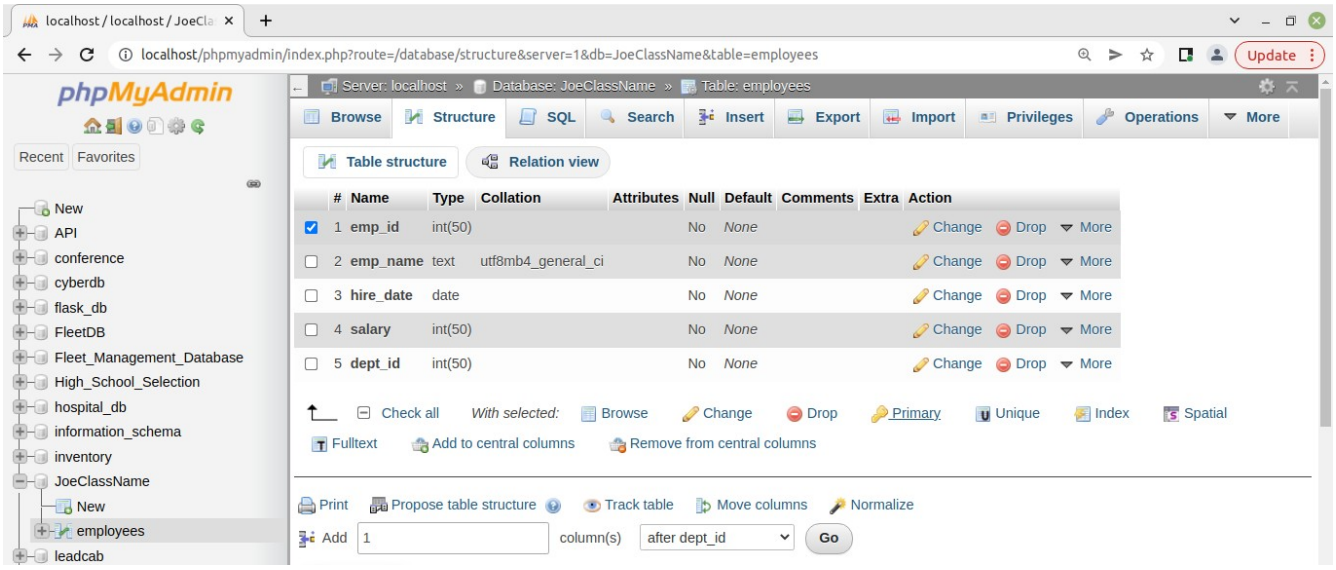


MODCOM Institute of Technology

Adding Primary Key.

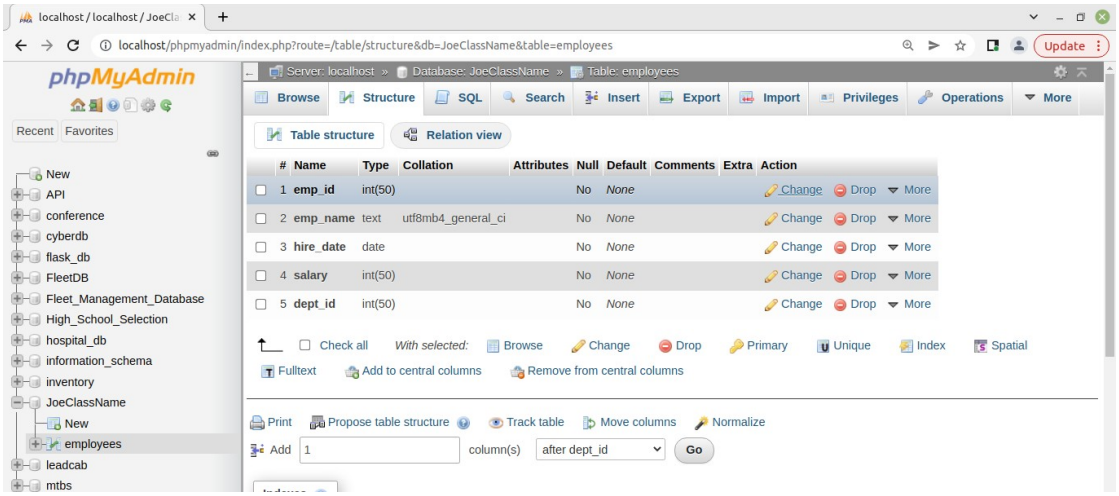
Here we need to select the most unique key from the table, I.e the column that cannot have duplicates, we select emp_id to be unique.

To do that Tick on emp_id and select **Primary, See below**

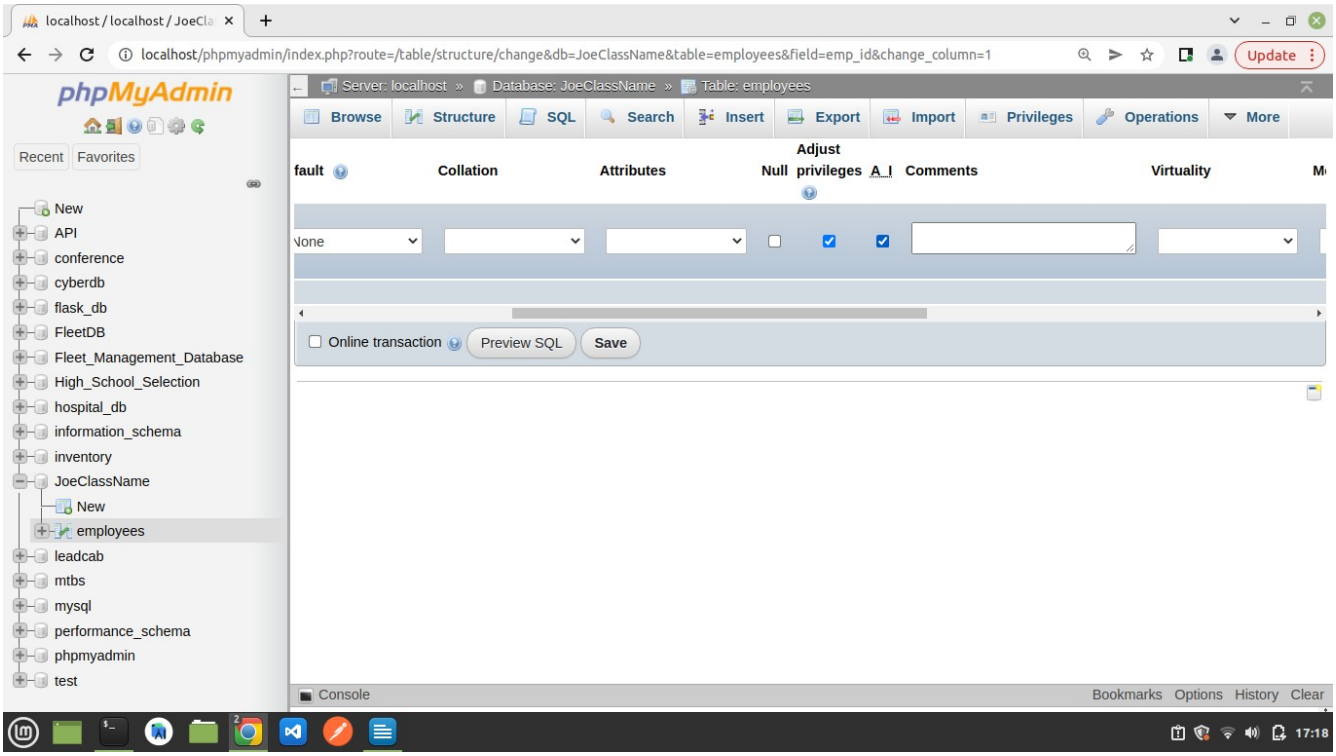


Adding Auto increment

To add an auto increment Field, Click on Change under **emp_id**, then tick on A_I Box.



Tick **A_I** Box



Now the table is now created.

SQL QUERY

SQL Insert

SQL Select

SQL Where

SQL AND & OR

SQL IN & Between

SQL Order By

SQL Top/Limit

SQL Distinct

SQL Update

SQL Delete

MODCOM Institute of Technology

Inserting Data in Table

In the preceding lesson we've created a table named **employees** in our *demo* database. Now it's time to insert some data inside our newly created database table.

The `INSERT INTO` statement is used to insert new rows in a database table.

SQL INSERT Statement

Syntax

The basic syntax for inserting data into a table can be given with:

```
INSERT INTO table_name (column1,column2,...) VALUES (value1,value2,...);
```

Here the *column1*, *column2*,..., etc. represents the name of the table columns, whereas the *value1*, *value2*,..., and so on represents the corresponding values for these columns.

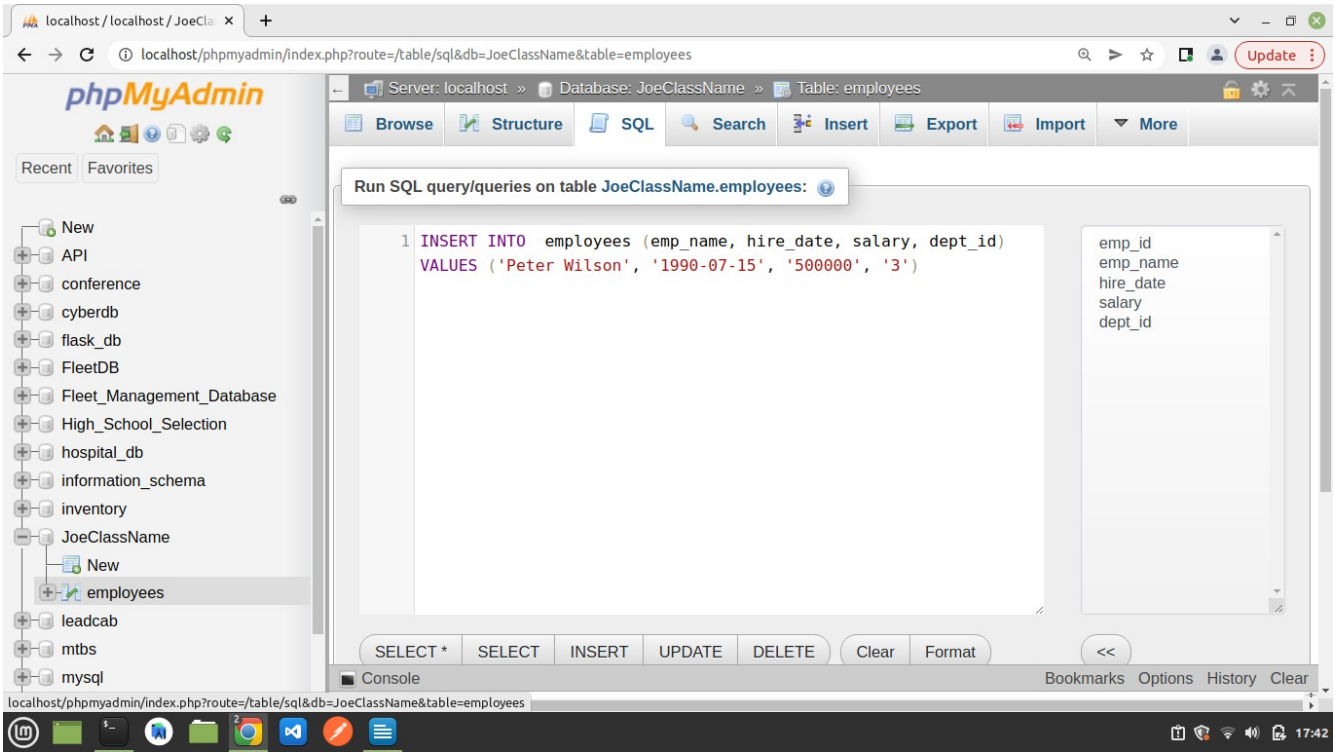
Let's insert some records into the **employees** table.

```
INSERT INTO  employees (emp_name, hire_date, salary, dept_id) VALUES ('Peter Wilson', '1990-07-15', '500000', '3');
```

Did you notice, we didn't insert any value for `id` field? Because, if you remember from the previous lesson the `id` field was marked with `AUTO_INCREMENT` flag, which tells MySQL to automatically assign a value to this field if it is left unspecified.

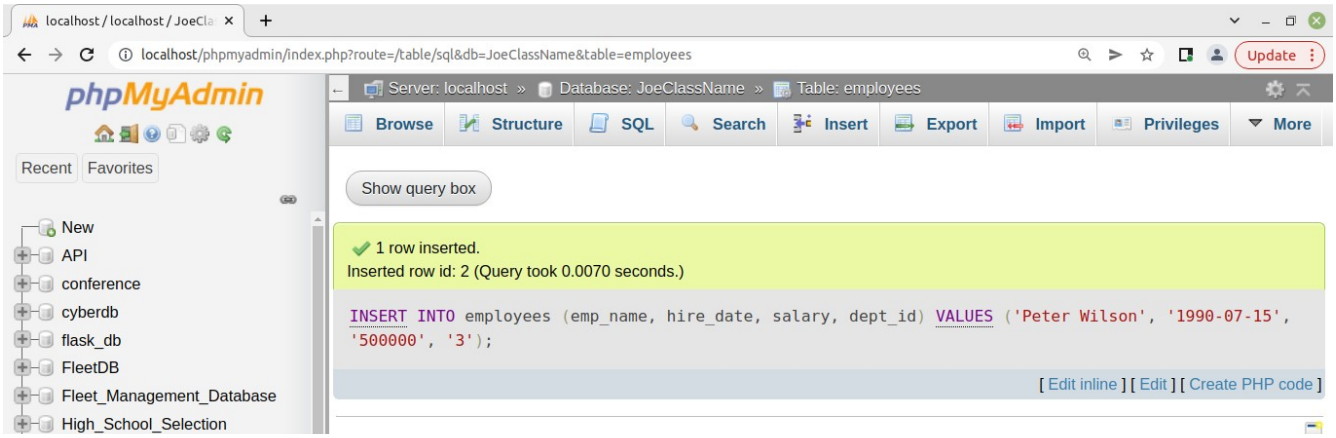
MODCOM Institute of Technology

Now to do practically on the table created earlier, On your database, make sure on you have clicked your table, the click on SQL.



Click on **Go** at bottom.

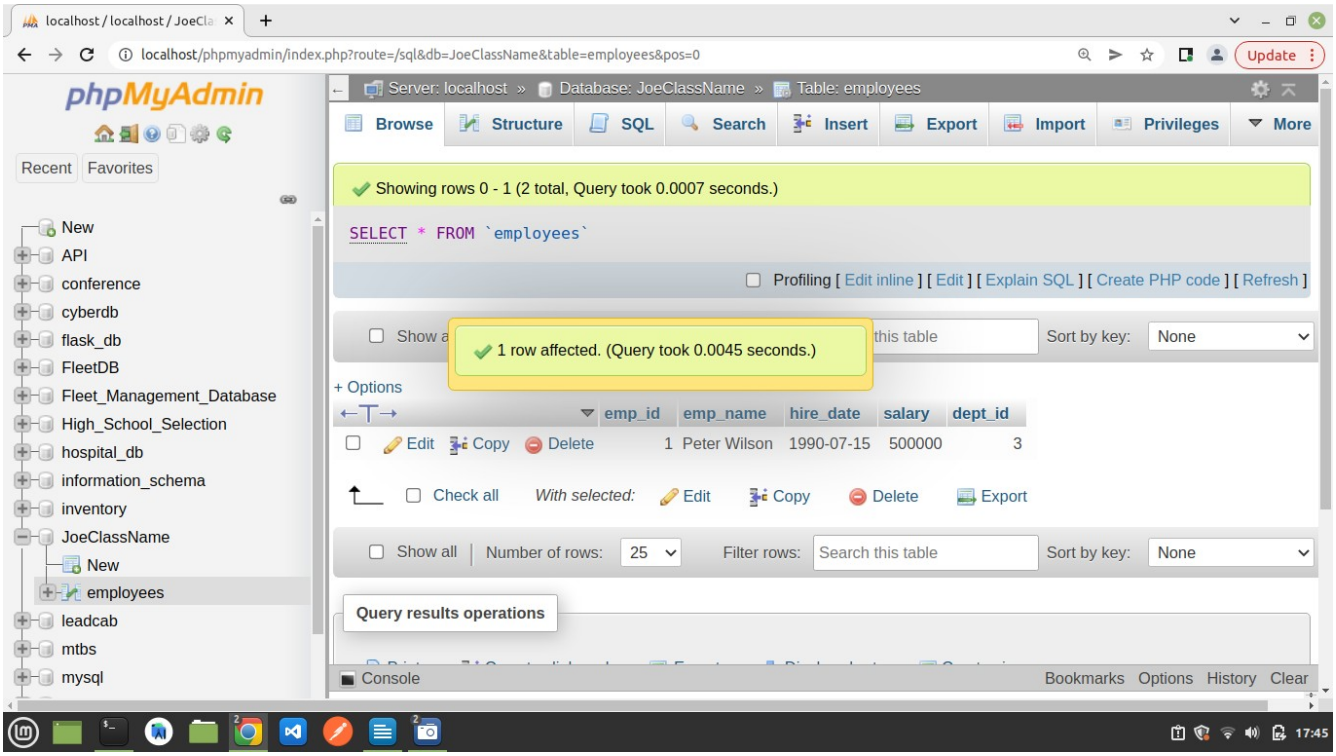
You'll see one row inserted.



MODCOM Institute of Technology

Click on **Browse** to see the data.

Below screenshot show the data inserted in out table.



Please insert another 5 records using below query

```
INSERT INTO  employees (emp_name, hire_date, salary, dept_id) VALUES ('Peter Wilson', '1990-07-15', '500000', '3');
```

SQL SELECT Statement

Selecting Data from Table

In the previous chapter we've learned how to insert data in a database table. Now it's time to select the data from existing tables using the SQL query.

The `SELECT` statement is used to select or retrieve the data from one or more tables. You can use this statement to retrieve all the rows from a table in one go, as well as to retrieve only those rows that satisfy a certain condition or a combination of conditions.

Syntax

The basic syntax for selecting the data from a table can be given with:

```
SELECT column1_name, column2_name, columnN_name FROM table_name;
```

Here, *column1_name*, *column2_name*, ... are the names of the columns or fields of a database table whose values you want to fetch. However, if you want to fetch the values of all the columns available in a table, you can just use the following syntax:

```
SELECT * FROM table_name;
```


MODCOM Institute of Technology

Let's put these statements into real use. Suppose we've a table named *employees* in our database that contains the following records:

```
+--------+--------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+--------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       4 |

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |

|      5 | Martin Blank | 2008-06-24 |   5600 |       6 |

+--------+--------------+------------+--------+---------+
```


MODCOM Institute of Technology

Select All from Table

The following statement will return all the rows from the *employees* table.

Example

```
SELECT * FROM employees;
```

After execution, the output will look something like this:

```
+--------+--------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+--------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       4 |

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |

|      5 | Martin Blank | 2008-06-24 |   5600 |       8 |

+--------+--------------+------------+--------+---------+
```

As you can see, it returns all the rows and columns from the *employees* table.

MODCOM Institute of Technology

Select Columns from Table

If you don't require all the data, you can select specific columns, like this:

Example

```
SELECT emp_id, emp_name, hire_date, salary
FROM employees;
```

After executing the above statement, you'll get the output something like this:

```
+--------+--------------+------------+--------+

| emp_id | emp_name     | hire_date  | salary |

+--------+--------------+------------+--------+

|      1 | Ethan Hunt   | 1995-10-30 |   5000 |

|      2 | Tony Montana | 1990-07-15 |   6500 |

|      3 | Sarah Connor | 2011-04-13 |   5600 |

|      4 | Rick Deckard | 2005-10-18 |   7200 |

|      5 | Martin Blank | 1996-05-24 |   8000 |

+--------+--------------+------------+--------+
```

As you can see this time there is no *dept_id* column in the result set. In the next chapter we'll learn how to select the records from a table based on a condition.

MODCOM Institute of Technology

SQL WHERE Clause

Selecting Record Based on Condition

In the previous chapter we've learnt how to fetch all the records from a table or table columns. But, in real world scenario we generally need to select, update or delete only those records which fulfill certain condition like users who belongs to a certain age group, or country, etc.

The `WHERE` clause is used with the SELECT, UPDATE, and DELETE. However, you'll see the use of this clause with other statements in upcoming chapters.

Syntax

The `WHERE` clause is used with the `SELECT` statement to extract only those records that fulfill specified conditions. The basic syntax can be given with:

```
SELECT column_list FROM table_name WHERE condition;
```

Here, *column_list* are the names of columns/fields like *name*, *age*, *country* etc. of a database table whose values you want to fetch. However, if you want to fetch the values of all the columns available in a table, you can use the following syntax:

```
SELECT * FROM table_name WHERE condition;
```

Now, let's check out some examples that demonstrate how it actually works.

Suppose we've a table called *employees* in our database with the following records:

MODCOM Institute of Technology

```
+--------+-------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+-------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       4 |

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |

|      5 | Martin Blank | 2008-06-24 |   5600 |       7 |

+--------+-------------+------------+--------+---------+
```

Filter Records with WHERE Clause

The following SQL statement will returns all the employees from the *employees* table, whose salary is greater than 7000. The `WHERE` clause simply filtered out the unwanted data.

Example

```
SELECT * FROM employees  WHERE salary > 7000;
```

After execution, the output will look something like this:

```
+--------+-------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |
```


MODCOM Institute of Technology

```
+--------+-------------+------------+--------+---------+

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |

+--------+-------------+------------+--------+---------+
```

As you can see the output contains only those employees whose salary is greater than 7000. Similarly, you can fetch records from specific columns, like this:

Example

```
SELECT emp_id, emp_name, hire_date, salary
FROM employees
WHERE salary > 7000;
```

After executing the above statement, you'll get the output something like this:

```
+--------+-------------+------------+--------+

| emp_id | emp_name     | hire_date  | salary |

+--------+-------------+------------+--------+

|      3 | Sarah Connor | 2005-10-18 |   8000 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |

+--------+-------------+------------+--------+
```

The following statement will fetch the records of an employee whose employee id is 2.

MODCOM Institute of Technology

Example

```
SELECT * FROM employees
WHERE emp_id = 2;
```

This statement will produce the following output:

```
+--------+-------------+-----------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+-------------+-----------+--------+---------+

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |

+--------+-------------+-----------+--------+---------+
```

This time we got only one row in the output, because *emp_id* is unique for every employee.

Operators Allowed in WHERE Clause

SQL supports a number of different operators that can be used in `WHERE` clause, the most important ones are summarized in the following table.

Operator	Description	Example
=	Equal	WHERE id = 2
>	Greater than	WHERE age > 30
<	Less than	WHERE age < 18
>=	Greater than or equal	WHERE rating >= 4
<=	Less than or equal	WHERE price <= 100

MODCOM Institute of Technology

Operator	Description	Example
LIKE	Simple pattern matching	WHERE name LIKE 'Dav'
IN	Check whether a specified value matches any value in a list or subquery	WHERE country IN ('USA', 'UK')
BETWEEN	Check whether a specified value is within a range of values	WHERE rating BETWEEN 3 AND 5

SQL AND & OR Operators

Selecting Record Based on Condition

In the previous chapter we've learned how to fetch records from a table using a single condition with the `WHERE` clause. But sometimes you need to filter records based on multiple conditions like selecting users whose ages are greater than 30 and country is United States, selecting products whose price is lower than 100 dollar and ratings is greater than 4, etc.

The AND Operator

The `AND` operator is a logical operator that combines two conditions and returns `TRUE` only if both condition evaluate to `TRUE` . The `AND` operator is often used in the WHERE clause of the SELECT, UPDATE, DELETE statement to form conditions to filter the result set.

```
SELECT column1_name, column2_name, columnN_name
```

```
FROM table_name
```


MODCOM Institute of Technology

```
WHERE condition1 AND condition2;
```

Let's check out some examples that demonstrate how it actually works.

Suppose we've a table called *employees* in our database with the following records:

```
+--------+--------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+--------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       4 |

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |

|      5 | Martin Blank | 2008-06-24 |   5600 |       7 |

+--------+--------------+------------+--------+---------+
```

Using WHERE Clause with AND Operator

The following SQL statement will return only those employees from the *employees* table whose salary is greater than 7000 and the *dept_id* is equal to 5.

MODCOM Institute of Technology

Example

```
SELECT * FROM employees
WHERE salary > 7000 AND dept_id = 5;
```

After execution, you will get the output something like this:

```
+--------+-------------+-----------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+-------------+-----------+--------+---------+

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

+--------+-------------+-----------+--------+---------+
```

The OR Operator

Similarly, the `OR` operator is also a logical operator that combines two conditions, but it returns `TRUE` when either of the conditions is `TRUE`.

The following SQL statement will return all the employees from the *employees* table whose salary is either greater than 7000 or the *dept_id* is equal to 5.

Example

```
SELECT * FROM employees
WHERE salary > 7000 OR dept_id = 5;
```

This time you will get the output something like this:

```
+--------+-------------+-----------+--------+---------+
```


MODCOM Institute of Technology

| emp_id | emp_name | hire_date | salary | dept_id |

+--------+-------------+-----------+--------+---------+

| 3 | Sarah Connor | 2005-10-18 | 8000 | 5 |

| 4 | Rick Deckard | 2007-01-03 | 7200 | 3 |

+--------+-------------+-----------+--------+---------+

Combining AND & OR Operator

You can also combine `AND` and `OR` to create complex conditional expressions.

The following SQL statement will return all the employees whose salary is greater than 5000 and the *dept_id* is either equal to 1 or 5.

Example

```
SELECT * FROM employees
WHERE salary > 5000 AND (dept_id = 1 OR dept_id = 5);
```

After executing the above query, you will get the output something like this:

+--------+-------------+-----------+--------+---------+

| emp_id | emp_name | hire_date | salary | dept_id |

+--------+-------------+-----------+--------+---------+

| 2 | Tony Montana | 2002-07-15 | 6500 | 1 |

| 3 | Sarah Connor | 2005-10-18 | 8000 | 5 |

MODCOM Institute of Technology

+--------+-------------+-----------+--------+---------+

SQL IN & BETWEEN Operators

Working with Range and Membership Conditions

In the previous chapter we've learned how to combine multiple conditions using the AND and OR operators. However, sometimes this is not sufficient and very productive, for example, if you have to check the values that lie within a range or set of values.

And here the IN and BETWEEN operators comes in picture that lets you define an exclusive range or a set of values rather than combining the separate conditions.

The IN Operator

The IN operator is logical operator that is used to check whether a particular value exists within a set of values or not. Its basic syntax can be given with:

```
SELECT column_list FROM table_name
```

```
WHERE column_name IN (value1, value1,...);
```

Here, *column_list* are the names of columns/fields like *name*, *age*, *country* etc. of a database table whose values you want to fetch. Well, let's check out some examples.

Consider we've an *employees* table in our database that has following records:

+--------+-------------+-----------+--------+---------+

| emp_id | emp_name | hire_date | salary | dept_id |

```
+--------+--------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       4 |

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |

|      5 | Martin Blank | 2008-06-24 |   5600 |       6 |

+--------+--------------+------------+--------+---------+
```

The following SQL statement will return only those employees whose *dept_id* is either 1 or 3.

Example

```
SELECT * FROM employees
WHERE dept_id IN (1, 3);
```

After executing the query, you will get the result set something like this:

```
+--------+--------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+--------------+------------+--------+---------+

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |
```


MODCOM Institute of Technology

```
+--------+-------------+------------+--------+---------+
```

Similarly, you can use the `NOT IN` operator, which is exact opposite of the `IN`. The following SQL statement will return all the employees except those whose *dept_id* is not 1 or 3.

Example

```
SELECT * FROM employees
WHERE dept_id NOT IN (1, 3);
```

After executing the query, this time you will get the result set something like this:

```
+--------+-------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+-------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       4 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |

+--------+-------------+------------+--------+---------+
```

The BETWEEN Operator

Sometimes you want to select a row if the value in a column falls within a certain range. This type of condition is common when working with numeric data.

To perform the query based on such condition you can utilize the `BETWEEN` operator. It is a logical operator that allows you to specify a range to test, as follow:

MODCOM Institute of Technology

```
SELECT column1_name, column2_name, columnN_name
FROM table_name
WHERE column_name BETWEEN min_value AND max_value;
```

Let's build and perform the queries based upon range conditions on our *employees* table.

Define Numeric Ranges

The following SQL statement will return only those employees from the *employees* table, whose salary falls within the range of 7000 and 9000.

Example

```
SELECT * FROM employees
WHERE salary BETWEEN 7000 AND 9000;
```

After execution, you will get the output something like this:

```
+--------+-------------+------------+--------+---------+
| emp_id | emp_name    | hire_date  | salary | dept_id |
+--------+-------------+------------+--------+---------+
|      3 | Sarah Connor | 2005-10-18 |   8000 |       5 |
|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |
+--------+-------------+------------+--------+---------+
```


MODCOM Institute of Technology

SQL ORDER BY Clause

Ordering the Result Set

Generally when you use the `SELECT` statement to fetch data from a table, the rows in result set are not in any particular order. If you want your result set in a particular order, you can specify the `ORDER BY` clause at the end of the statement which tells the server how to sort the data returned by the query. The default sorting order is ascending.

Syntax

The `ORDER BY` clause is used to sort the data returned by a query in ascending or descending order. The basic syntax of this clause can be given with:

```
SELECT column_list FROM table_name ORDER BY column_name ASC|DESC;
```

Here, *column_list* are the names of columns/fields like *name*, *age*, *country* etc. of a database table whose values you want to fetch, whereas the *column_name* is name of the column you want to sort. Let's check out some examples that demonstrate how it actually works.

Consider we've an *employees* table in our database that has following records:

```
+--------+--------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+--------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       4 |

|      2 | Tony Montana | 2002-07-15 |   6500 |       1 |
```


MODCOM Institute of Technology

| 3 | Sarah Connor | 2005-10-18 | 8000 | 5 |

| 4 | Rick Deckard | 2007-01-03 | 7200 | 3 |

| 5 | Martin Blank | 2008-06-24 | 5600 | 7 |

+--------+--------------+------------+--------+---------+

Sorting Single Column

The following SQL statement will return all the employees from the *employees* table and orders the result set by the *emp_name* column in ascending order.

Example

```
SELECT * FROM employees
ORDER BY emp_name ASC;
```

You can skip the `ASC` option and simply use the following syntax. It returns the same result set as previous statement, because the default sorting order is ascending:

Example

```
SELECT * FROM employees
ORDER BY emp_name;
```

After executing the above command, you'll get the output something like this:

+--------+--------------+------------+--------+---------+

| emp_id | emp_name | hire_date | salary | dept_id |

+--------+--------------+------------+--------+---------+

MODCOM Institute of Technology

```
|      1 | Ethan Hunt   | 2001-05-01 |   5000 |      4 |

|      5 | Martin Blank | 2008-06-24 |   5600 |    NULL |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |      5 |

|      2 | Tony Montana | 2002-07-15 |   6500 |      1 |

+--------+--------------+------------+--------+---------+
```

Similarly, you can use the `DESC` option to perform a sorting in descending order. The following statement will orders the result set by the numeric *salary* column in descending order.

Example

```
SELECT * FROM employees
ORDER BY salary DESC;
```

This time, you'll get the result set something like this:

```
+--------+--------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+--------------+------------+--------+---------+

|      3 | Sarah Connor | 2005-10-18 |   8000 |      5 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       3 |
```


MODCOM Institute of Technology

```
|      2 | Tony Montana | 2002-07-15 |   6500 |      1 |

|      5 | Martin Blank | 2008-06-24 |   5600 |    NULL |

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |      4 |

+--------+--------------+------------+--------+---------+
```

SQL UPDATE Statement

Updating Table Data

In the previous lessons we've learnt how to insert data as well as how to select data from a database table based on various conditions. In this tutorial we'll learn to perform one more important task which is updating the existing records in a database table.

Syntax

The `UPDATE` statement is used to update existing data in a table.

```
UPDATE table_name

SET column1_name = value1, column2_name = value2,...

WHERE condition;
```


MODCOM Institute of Technology

Here, *column1_name*, *column2_name*,... are the names of the columns or fields of a database table whose values you want to update. You can also combine multiple conditions using the AND or OR operators, that you've learned in the previous chapters.

> Warning: The WHERE clause in the UPDATE statement specifies which record or records should be updated. If you omit the WHERE clause, all the records will be updated.

Let's check out some examples that demonstrate how it actually works.

Suppose we've an *employees* table in our database that has following records:

```
+--------+--------------+------------+--------+---------+

| emp_id | emp_name     | hire_date  | salary | dept_id |

+--------+--------------+------------+--------+---------+

|      1 | Ethan Hunt   | 2001-05-01 |   5000 |       1 |

|      2 | Tony Montana | 2002-07-15 |   6500 |       5 |

|      3 | Sarah Connor | 2005-10-18 |   8000 |       3 |

|      4 | Rick Deckard | 2007-01-03 |   7200 |       4 |

|      5 | Martin Blank | 2008-06-24 |   5600 |    NULL |
```


MODCOM Institute of Technology

Updating a Single Column

The following SQL statement will update the *emp_name* field of the *employees* table and set a new value where the employee id i.e. *emp_id* is equal to 3.

Example

```
UPDATE employees SET emp_name = 'Sarah Ann Connor'
WHERE emp_id = 3;
```

After execution, the resulting table will look something like this:

```
+--------+-----------------+------------+--------+---------+

| emp_id | emp_name        | hire_date  | salary | dept_id |

+--------+-----------------+------------+--------+---------+

|      1 | Ethan Hunt      | 2001-05-01 |   5000 |       1 |

|      2 | Tony Montana    | 2002-07-15 |   6500 |       5 |

|      3 | Sarah Ann Connor | 2005-10-18 |   8000 |       3 |

|      4 | Rick Deckard    | 2007-01-03 |   7200 |       4 |

|      5 | Martin Blank    | 2008-06-24 |   5600 |    NULL |

+--------+-----------------+------------+--------+---------+
```

MODCOM Institute of Technology

Updating Multiple Columns

Similarly, you can update multiple columns using a list of comma-separated column name and value pair. The following example will update the *salary* and *dept_id* field of an existing employee in the *employees* table whose *emp_id* is 5.

Example

```
UPDATE employees
SET salary = 6000, dept_id = 2
WHERE emp_id = 5;
```

After execution, the resulting table will look something like this:

```
+--------+-----------------+------------+--------+---------+

| emp_id | emp_name        | hire_date  | salary | dept_id |

+--------+-----------------+------------+--------+---------+

|      1 | Ethan Hunt      | 2001-05-01 |   5000 |       1 |

|      2 | Tony Montana    | 2002-07-15 |   6500 |       5 |

|      3 | Sarah Ann Connor | 2005-10-18 |   8000 |    3 |

|      4 | Rick Deckard    | 2007-01-03 |   7200 |      4 |

|      5 | Martin Blank    | 2008-06-24 |   6000 |       2 |

+--------+-----------------+------------+--------+---------+
```


MODCOM Institute of Technology

SQL DELETE Statement

Deleting Data from Tables

Just as you insert records into a table with the INSERT statement, you can delete records from a table as well with the DELETE statement.

Syntax

The DELETE statement is used to remove one or more rows from a table.

```
DELETE FROM table_name WHERE condition;
```

Warning: The WHERE clause in the DELETE statement specifies which record or records should be deleted. It is however optional, but if you omit or forget the WHERE clause, all the records will be deleted permanently from the table.

Let's delete some records from the *employees* table that we've created

Suppose that our *employees* table currently has the following records:

```
+--------+-----------------+------------+--------+---------+

| emp_id | emp_name        | hire_date  | salary | dept_id |

+--------+-----------------+------------+--------+---------+

|      1 | Ethan Hunt      | 2001-05-01 |   5000 |       1 |

|      2 | Tony Montana    | 2002-07-15 |   6500 |       5 |

|      3 | Sarah Ann Connor | 2005-10-18 |   8000 |    3 |
```


MODCOM Institute of Technology

| 4 | Rick Deckard | 2007-01-03 | 7200 | 4 |

| 5 | Martin Blank | 2008-06-24 | 6000 | 2 |

+--------+-----------------+------------+--------+---------+

Delete Records Based on Conditions

The following statement will delete the rows from *persons* table where *id* is greater than 3.

Example

```
DELETE FROM employees WHERE emp_id = 3;
```

After executing the query, the *employees* table will look something like this:

+--------+-----------------+------------+--------+---------+

| emp_id | emp_name | hire_date | salary | dept_id |

+--------+-----------------+------------+--------+---------+

| 1 | Ethan Hunt | 2001-05-01 | 5000 | 1 |

| 2 | Tony Montana | 2002-07-15 | 6500 | 5 |

| 4 | Rick Deckard | 2007-01-03 | 7200 | 4 |

| 5 | Martin Blank | 2008-06-24 | 6000 | 2 |

+--------+-----------------+------------+--------+---------+

Sarah Ann, with id number 3 is deleted

MODCOM Institute of Technology

Delete All Data

Similarly, as mentioned above if you do not specify the `WHERE` clause in the `DELETE` statement all the rows from the table will be deleted. However, the target table itself won't be deleted that means the table structure, attributes, and indexes will remain intact.

The following statement will remove all the records from the *persons* table:

Example

```
DELETE FROM employees;
```

This deletes all records in the employees table

MODCOM Institute of Technology

References

https://www.geeksforgeeks.org/sql-tutorial/

https://www.w3schools.com/sql/sql_intro.asp

https://aws.amazon.com/what-is/sql/